



Microprocessor & Computer Architecture (μpCA)

UE24CS251B

Session -
4



Microprocessor & Computer Architecture (μpCA)

UE24CS251B

Data Processing
Instructions

ARM instructions are grouped as follows.

1. Data Processing Instructions.

Ex: MOV, ADD, SUB, CMP, CMN, etc.,

2. Branch Instructions.

Ex: B, BL, BGT, BLT, BGE, BLE, etc.,

3. Data Transfer Instructions.

Ex: LOAD – STORE Instructions- LDR, STR, LDM, STM, etc.,

4. Software Interrupt Instructions.

Ex: SWI 0x11, etc.,

5. Program Status Register Instructions.

Ex: MSR, MRS, etc.,

R0 = negat e(R1)

Arithmetic Instruction:

Syntax: **<memop><cond><s> <dest Rd>, <src op1 Rn>, <src op2 Rm>**

Ex 3: **ADD** **R3,** **R1,**

R2

R3 **=** **R1** **+** **R2**

Adds the contents of the source operand registers R1 & R2 and stores the result in the destination register R3.

Compare Instruction:

Syntax:

Ex4: **CMP** **R3**, **R1**

Neither source nor destination fields are affected.

Ex 4: CMN R3,

- Any data processing instruction can set the condition codes if the programmers wish it to

64-bit addition

ADD^S R2, R2, R0

ADC R3, R3, R1

Comparison

compare

- **CMP** R1, R2 @ set cc on R1-R2

compare negated

- **CMN** R1, R2 @ set cc on R1+R2

bit

- **TST** R1, R2 @ set cc on R1 and R2

test

- **TEQ** R1, R2 @ set cc on R1 xor R2

- **MOV** R4, #10
- **MOV** R5, R4
- **MOV** R1, #5
- **MOV** R2, #2

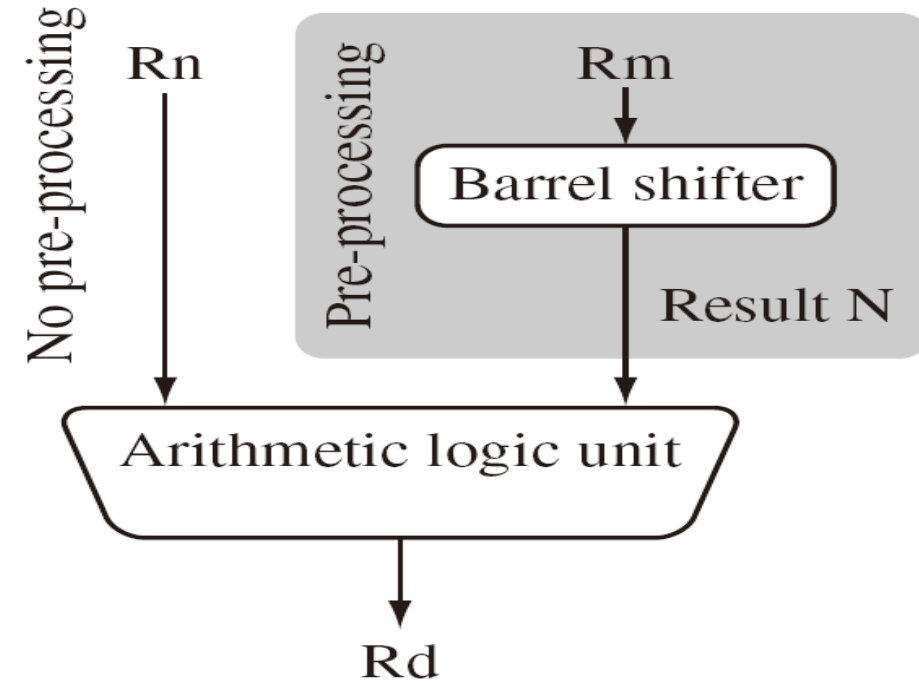
- **ADD** r0, r1, r2 ; r0: =r1+r2
- **ADC** r0, r1, r2 ; r0: =r1+r2+C
- **SUB** r0, r1, r2 ; r0: =r1-r2
- **SBC** r0, r1, r2 ; r0: =r1-r2+C-1
- **RSB** r0, r1, r2 ; r0: =r2-r1, **Reverse Subtraction**
- **RSC** r0, r1, r2 ; r0: =r2-r1+C-1

By default data processing operations do not affect the condition flags

- **AND** r_0, r_1, r_2 ; $r_0 := r_1$ **AND** r_2
- **ORR** r_0, r_1, r_2 ; $r_0 := r_1$ **OR** r_2
- **EOR** r_0, r_1, r_2 ; $r_0 := r_1$ **XOR** r_2
- **BIC** r_0, r_1, r_2 ; $r_0 := r_1$ **AND** (**NOT**
 r_2), bit clear

Shifted register operands

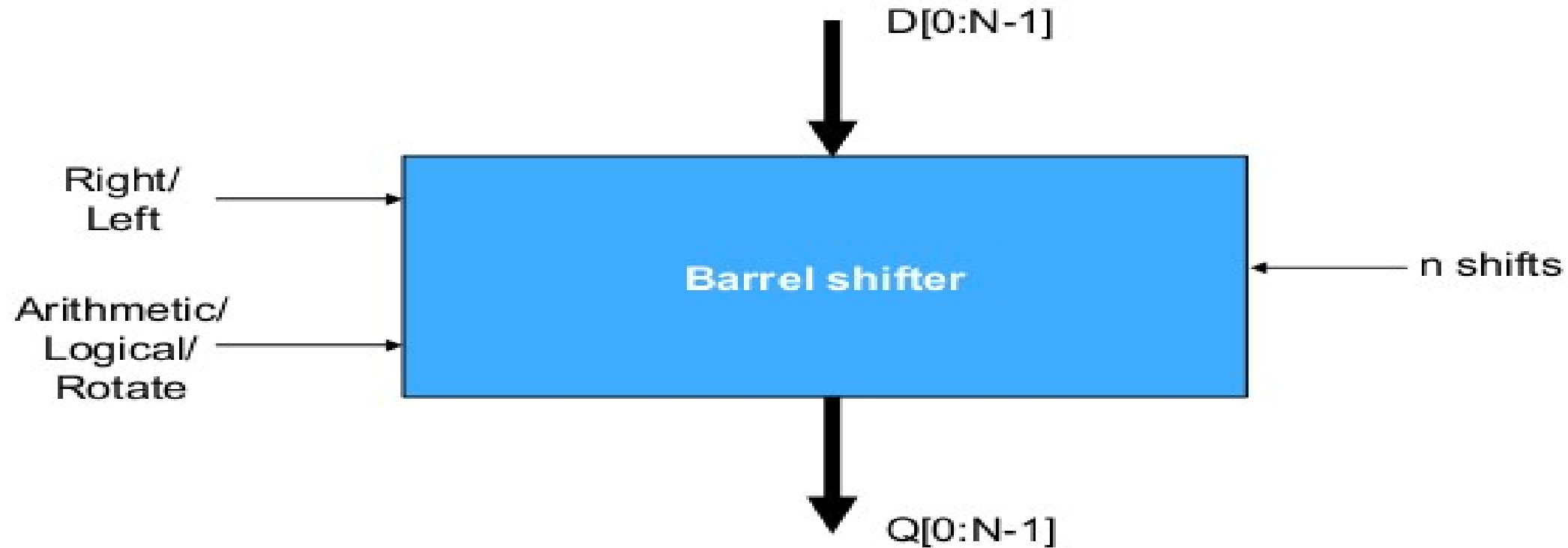
- One operand to ALU is routed through the **Barrel shifter**. Thus, the operand can be modified before it is used. Useful for fast multiplication and dealing with lists, table and other complex data structure. (similar to the displacement addressing mode in CISC.)



- Some instructions (e.g. **MUL**, **CLZ**) do not read barrel shifter.

BARREL SHIFTER - BLOCK DI AGRAM

Barrel shifter



SHIFTER

- ARM architecture supports instructions which perform a shift with ALU operation.
- The shifter performance is therefore critical as it contributes directly to the datapath cycle time.
- To minimize the delay through the shifter, A cross-bar switch matrix is used to steer each input to appropriate output.
- Example shown is 4 x 4 matrix cross bar switch.
- ARM uses a 32 x 32 matrix cross bar switch.
- Each input is connected to each output through a switch.

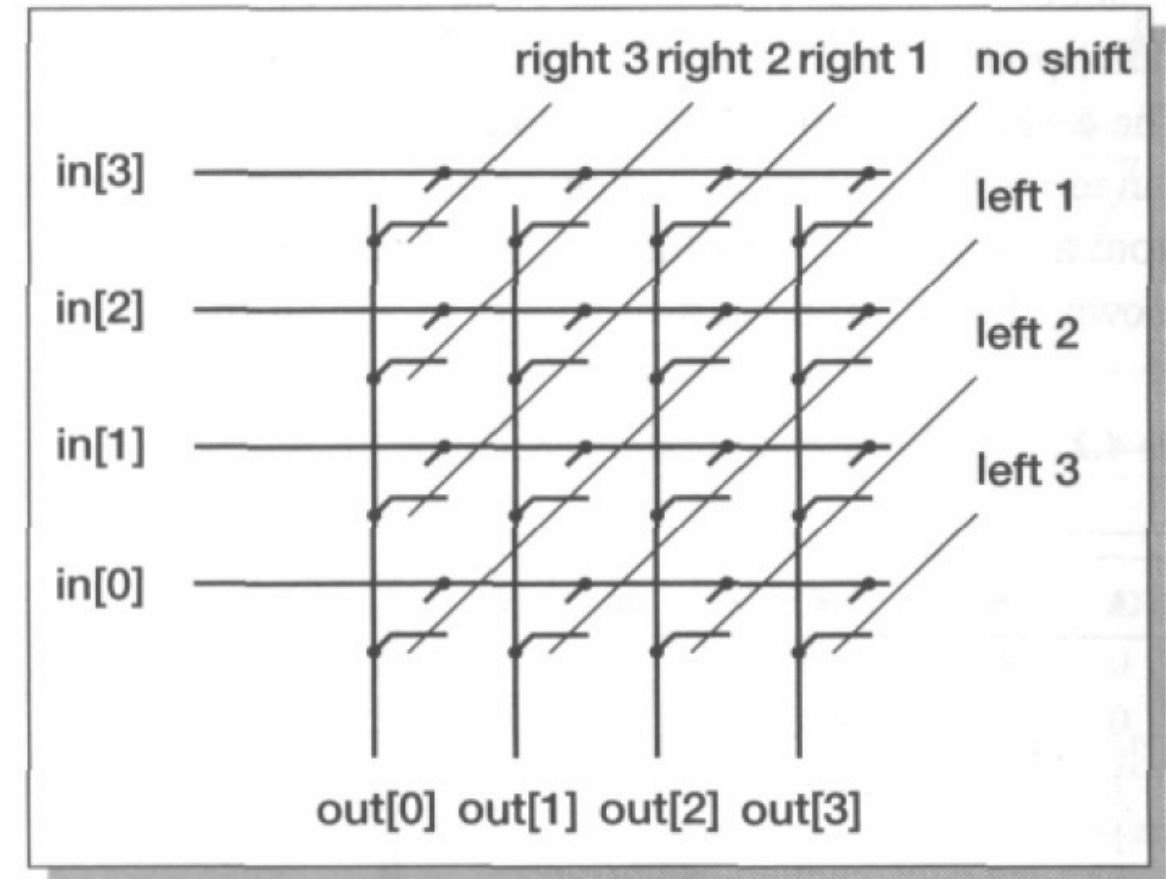


Figure 4.15 The cross-bar switch barrel shifter principle.

BARREL SHIFTER - FUNCTION

- Shift Operation – Left
 - One left shift diagonal line is turned on.
- Shift Operation – Right
 - One right shift diagonal line is turned on.
- Arithmetic Shift right
 - Uses sign extension rather than zero filling for unconnected output.
- Rotate operation
 - The right shift diagonal is enabled together with complementary left shift diagonal.

Mnemonic	Description	Shift	Result
LSL	logical shift left	$x\text{LSL } y$	$x \ll y$
LSR	logical shift right	$x\text{LSR } y$	$(\text{unsigned})x \gg y$
ASR	arithmetic right shift	$x\text{ASR } y$	$(\text{signed})x \gg y$
ROR	rotate right	$x\text{ROR } y$	$((\text{unsigned})x \gg y) (x \ll (32 - y))$
RRX	rotate right extended	$x\text{RRX}$	$(c\text{ flag} \ll 31) ((\text{unsigned})x \gg 1)$

MULTIPLICATION BY CONSTANT USING THE BARREL SHIFTER

- Multiplication by 2^n (1, 2, 4, 8, 16, 32..)

MOV Ra, Rb, LSL #n

- Multiplication by 2^{n+1} (3, 5, 9, 17..)

ADD Ra, Ra, Ra, LSL #n

- Multiplication by 2^{n-1} (3, 7, 15..)

RSB Ra, Ra, Ra, LSL #n

- Multiplication by 6

ADD Ra, Ra, Ra, LSL #1 ; multiply by 3

MOV Ra, Ra, LSL#1 ; and then by 2

- Multiply by 10 and add in extra number

ADD Ra, Ra, Ra, LSL#2 ; multiply by 5

ADD Ra, Rc, Ra, LSL#1 ; multiply by 2 and add in next

digit

BARREL SHIFTER - EXAMPLES

- Shift Operation – Left
 - One left shift diagonal line is turned on.

Example 1: ADD R0, R1, R2,
LSL #3

- Since LSL is used, it selects left diagonal #3.
- Rotate operation
 - The right shift diagonal is enabled together with complementary left shift diagonal.

Example 2: ADD R0, R1, R2,
ROR #3

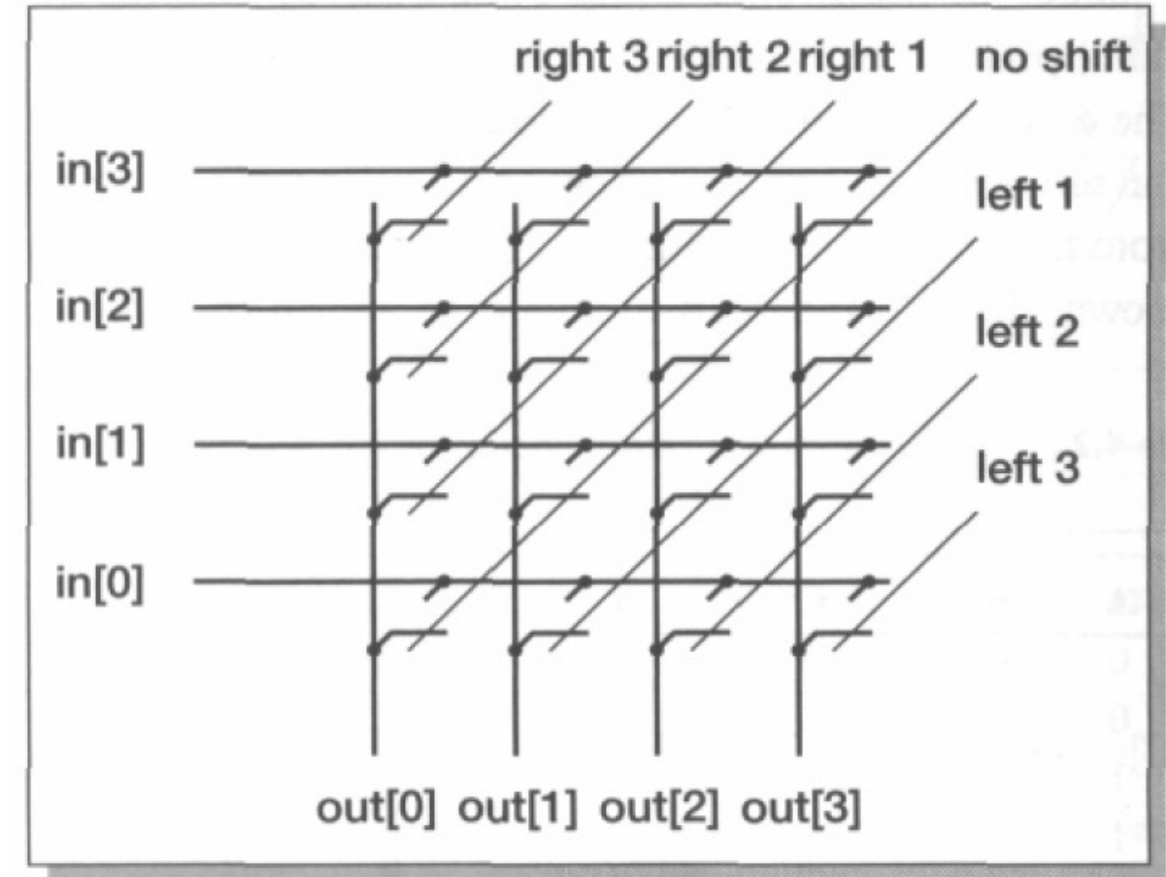
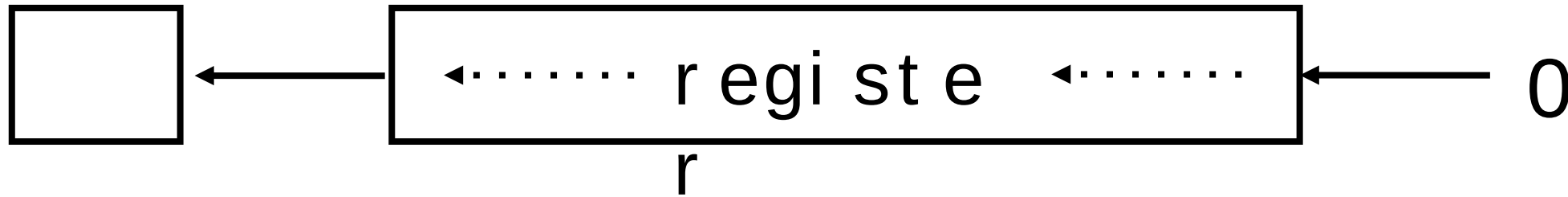


Figure 4.15 The cross-bar switch barrel shifter principle.

Logical shift left



MOV R0, R2, **LSL #2** @ R0:=R2<<2
@ R2 unchanged

Example: **0...0 0011 0000**

Before R2=0x00000030

After R0=0x000000C0

R2=0x00000030

Logical shift right



MOV R0, R2, **LSR #2** @ R0:=R2>>2
@ R2 unchanged

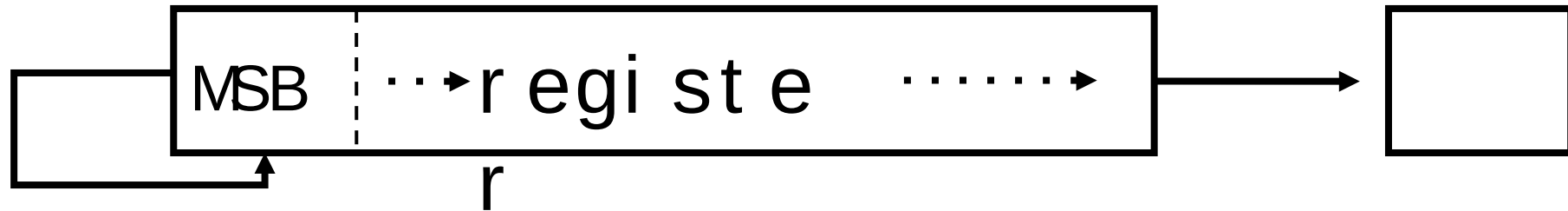
Example: **0...0 0011 0000**

Before R2=0x00000030

After R0=0x0000000C

R2=0x00000030

Arithmetic shift right



MOV R0, R2, **ASR #2** @ R0:=R2>>2
@ R2 unchanged

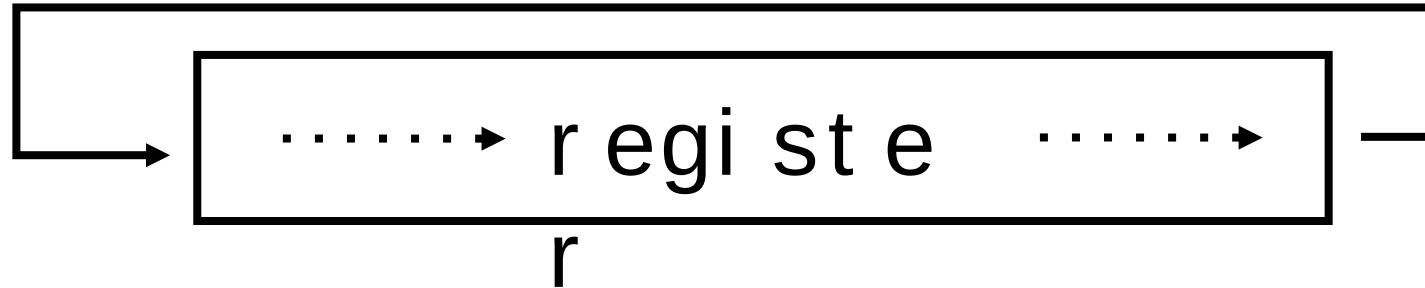
Example: **1010 0...0 0011 0000**

Before R2=0xA0000030

After R0=0xE800000C

R2=0xA0000030

Rot at e r i g h t



MOV R0, R2, ROR #2 @ R0:=R2 rotate
@ R2 unchanged

Example: 0...0 0011 0001

Before R2=0x00000031

After R0=0x4000000C

R2=0x00000031

Rot at e r i g h t e x t e n d e d



MOV R0, R2, **RRX** @ R0:=R2 rotate
 @ R2 unchanged

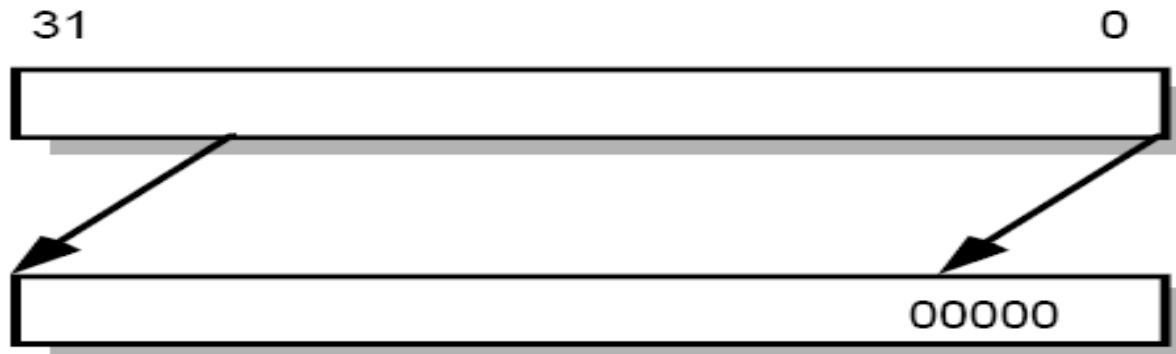
Example: 0...0 0011 0001

Before R2=0x00000031, C=1

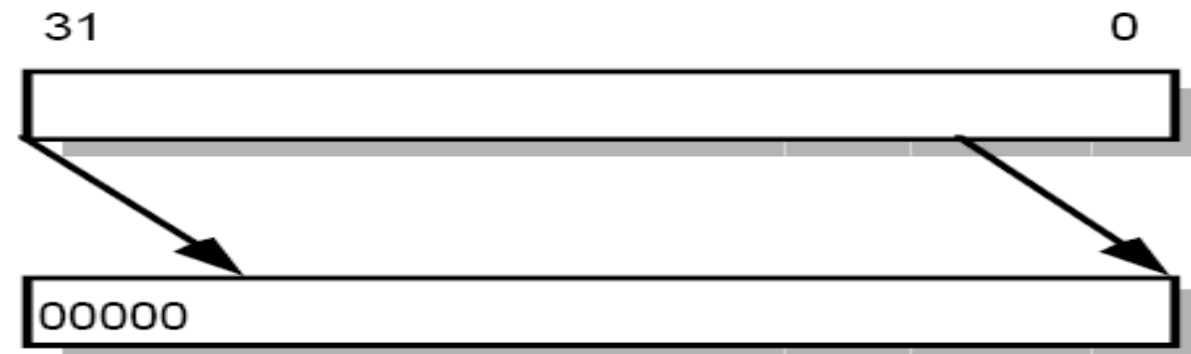
After R0=0x80000018, C=1

R2=0x00000031

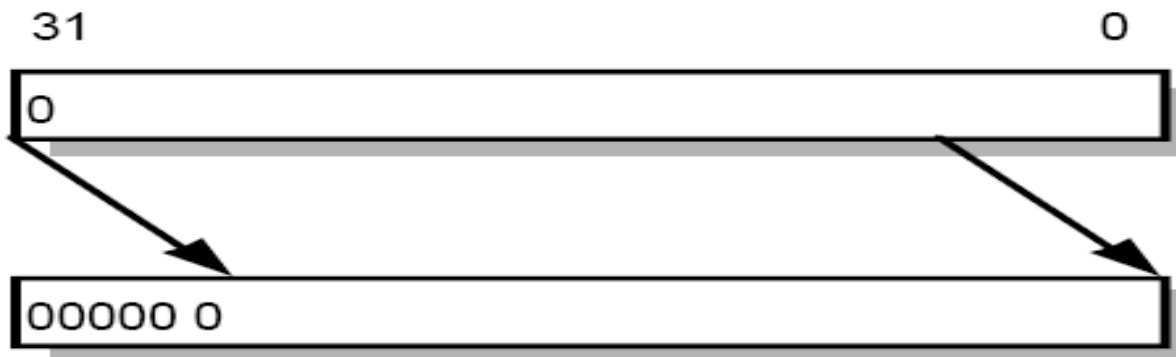
Shifted register operands



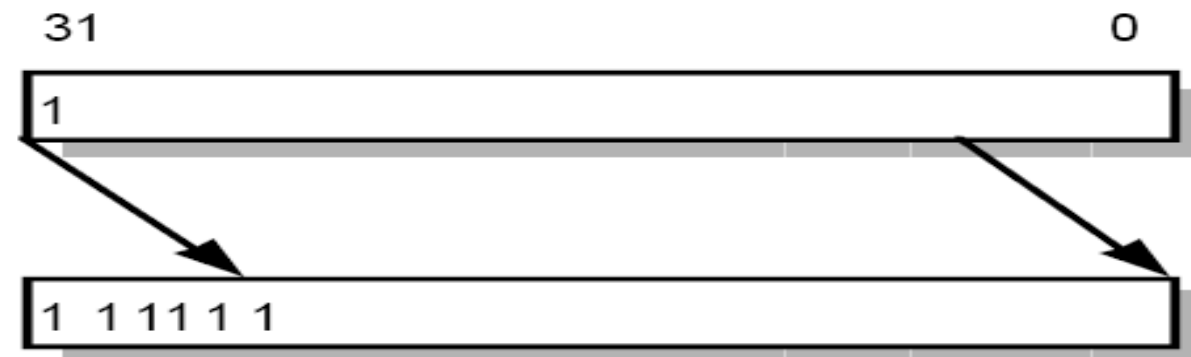
LSL #5



LSR #5

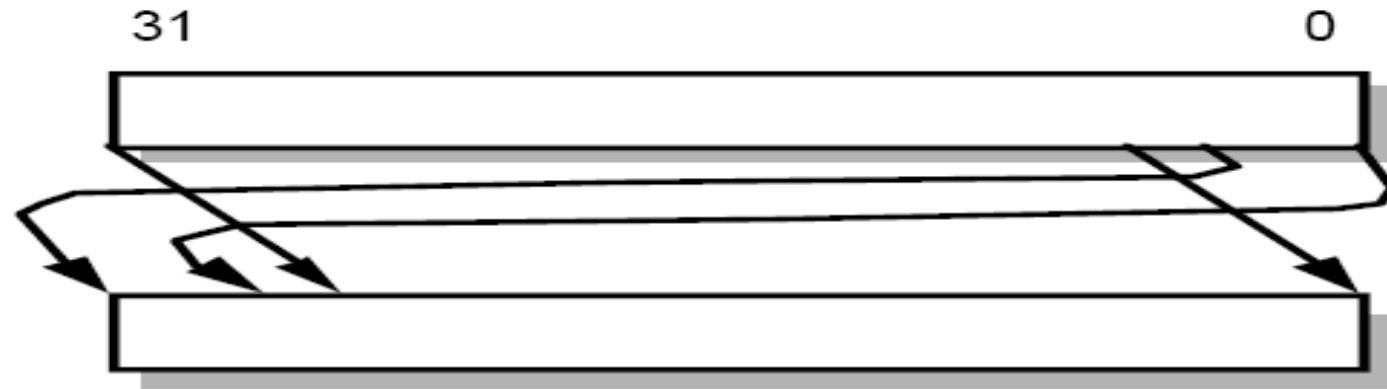


ASR #5 , positive operand

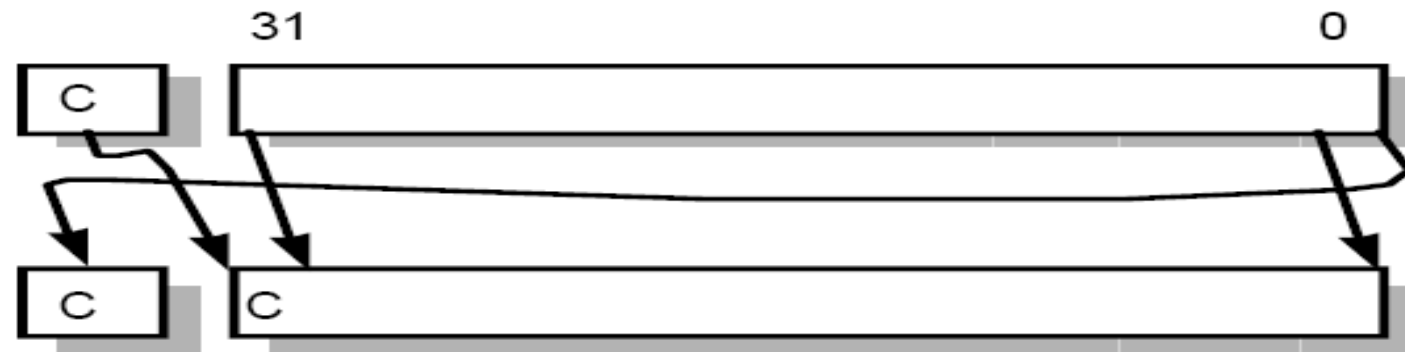


ASR #5 , negative operand

Shifted register operands



ROR #5



RRX

- It is possible to use a register to specify the number of bits to be shifted; only the bottom 8 bits of the register are significant.

@ array index calculation

ADD R0, R1, R2, LSL R3 @ $R0 := R1 + R2 * 2^{R3}$

@ fast multiply $R2 = 35 \times R0$

ADD R0, R0, R0, LSL #2 @ $R0' = 5 \times R0$

RSB R2, R0, R0, LSL #3 @ $R2 = 7 \times R0'$

Shifted register operands

<i>N</i> shift operations	Syntax
Immediate	#immediate
Register	Rm
Logical shift left by immediate	Rm, LSL #shift_imm
Logical shift left by register	Rm, LSL Rs
Logical shift right by immediate	Rm, LSR #shift_imm
Logical shift right with register	Rm, LSR Rs
Arithmetic shift right by immediate	Rm, ASR #shift_imm
Arithmetic shift right by register	Rm, ASR Rs
Rotate right by immediate	Rm, ROR #shift_imm
Rotate right by register	Rm, ROR Rs
Rotate right with extend	Rm, RRX

MOV R1, #35

MUL R2, R0, R1

or

ADD R0, R0, R0, LSL #2 @ $R0' = 5 \times R0$

RSB R2, R0, R0, LSL #3 @ $R2 = 7 \times R0'$

1. Which of the following best describes the purpose of data processing instructions in the ARM instruction set?
 - A. They control program flow using branch operations
 - B. They perform arithmetic, logical, and comparison operations on register data
 - C. They transfer data directly between memory locations
 - D. They manage processor modes and interrupts

2. Which of the following statements about comparison instructions in ARM is correct?
 - A. They store the result of comparison in a destination register
 - B. They modify only the source registers
 - C. They update condition flags without storing the result
 - D. They perform memory-to-memory comparison

3. Which instruction uses the barrel shifter to multiply a register value by 2^n in a single ARM instruction?

- A. ADD Ra, Ra, #n
- B. MOV Ra, Rb, LSL #n
- C. MUL Ra, Rb, #n
- D. RSB Ra, Rb, #n

4. Which shift operation in ARM preserves the sign bit while shifting bits to the right?

- A. Logical Shift Left (LSL)
- B. Logical Shift Right (LSR)
- C. Arithmetic Shift Right (ASR)
- D. Rotate Right (ROR)

1. Which of the following best describes the purpose of data processing instructions in the ARM instruction set?

- A. They control program flow using branch operations
- B. They perform arithmetic, logical, and comparison operations on register data
- C. They transfer data directly between memory locations
- D. They manage processor modes and interrupts

Answer - They perform arithmetic, logical, and comparison operations on register data

2. Which of the following statements about comparison instructions in ARM is correct?

- A. They store the result of comparison in a destination register
- B. They modify only the source registers
- C. They update condition flags without storing the result

3. Which instruction uses the barrel shifter to multiply a register value by 2^n in a single ARM instruction?

- A. ADD Ra, Ra, #n
- B. MOV Ra, Rb, LSL #n
- C. MUL Ra, Rb, #n
- D. RSB Ra, Rb, #n

Answer - MOV Ra, Rb, LSL #n

4. Which shift operation in ARM preserves the sign bit while shifting bits to the right?

- A. Logical Shift Left (LSL)
- B. Logical Shift Right (LSR)
- C. Arithmetic Shift Right (ASR)
- D. Rotate Right (ROR)



Arm Instruction Set – what a processing
group

Using the ARM instruction set, write Assembly level language instructions / sequence / programs to perform the following as directed.

1. Compile the following C assignment statements into ARM

i. $A = B + C;$

ii. $D = A - C;$

iii. Perform $D = A - C$ using RSB instruction.

2. Convert the following assignment into ARM

i. $F = (G + H) - (I + J).$

use the register R0 to R4 as operands F to J respectively.

3. Convert the statement into ARM where the operand is in memory.

i. $G = H + A[10].$

Assignment.

Note: Use different addressing modes both at source & destination operand positions.

THANK YOU



Team MPCA

Department of Computer Science
and Engineering